FastAPI Project

Code Explanation: Item Schema, Item Model, Router and Database Setup

Repository: https://github.com/codershalini210/FastAPIProject

A beginner-friendly guide explaining how the item API is structured, what each line does, how a request moves through the application, and how to test the endpoint.

Main concepts covered

• Pydantic schemas for request validation and response formatting

• SQLAlchemy model for the database table

• APIRouter and the POST endpoint

• Database engine, sessions and dependency injection

• How main.py connects the router to the FastAPI application

• Testing the endpoint using Swagger UI and example JSON

1. How the item feature is organised

The item feature separates responsibilities into different files. The schema describes the data the API accepts and returns. The SQLAlchemy model describes the database table. The router defines the URL and the operations to perform. The database module creates connections and supplies a session.

File	Purpose
models/itemSchema.py	Defines ItemCreate and ItemResponse using Pydantic.
models/itemModel.py	Defines the SQLAlchemy Item table and its columns.
routers/itemsRouter.py	Defines the POST /item endpoint and database operations.
models/database.py	Creates the SQLite engine, session factory, Base and get_db dependency.
main.py	Creates the FastAPI app, creates tables and registers routers.

2. Item schema — models/itemSchema.py

```
from pydantic import BaseModel, ConfigDict

class ItemCreate(BaseModel):
    name: str
    price: int
    in_stock: bool

class ItemResponse(BaseModel):
    id: int
    name: str
    price: int
    in_stock: bool
    model_config = ConfigDict(from_attributes=True)
```

from pydantic import BaseModel, ConfigDict

BaseModel is the base class used to define Pydantic data schemas. FastAPI uses these schemas to validate incoming JSON and to describe the expected data. ConfigDict provides configuration options for a Pydantic model.

class ItemCreate(BaseModel)

ItemCreate describes the data required when a client creates an item. The fields are name (text), price (integer), and in_stock (true/false). There is no id field because the database generates the primary-key id when a new record is inserted.

class ItemResponse(BaseModel)

ItemResponse describes the data the API sends back after creation. It includes id as well as the three item fields. This lets the API return the new database ID to the client.

model_config = ConfigDict(from_attributes=True)

This allows Pydantic to build the response schema from an object's attributes, such as a SQLAlchemy Item object. Without this setting, Pydantic v2 may expect dictionary-like input rather than reading attributes from an ORM object.

Important: the schema validates the shape and types of the data; it does not create a database table or save a record. Those jobs are handled elsewhere.

3. Item database model — models/itemModel.py

```
from sqlalchemy import Column, Integer, String, Boolean
from models.database import Base
```

```
class Item(Base):
    __tablename__ = "items"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    price = Column(Integer, nullable=False)
    in_stock = Column(Boolean, nullable=False)
```

Code	Meaning
Column, Integer, String, Boolean	SQLAlchemy column and data-type definitions.
Base	The shared declarative base from database.py; SQLAlchemy uses it to track mapped tables.
class Item(Base)	Declares a Python class mapped to a database table.
__tablename__ = "items"	Sets the database table name to items.
id = Column(Integer, primary_key=True, index=True)	Integer primary key; each item has a unique ID. index=True asks the database to index this column
name = Column(String, nullable=False)	Text value that must not be NULL.
price = Column(Integer, nullable=False)	Integer price value that must not be NULL.
in_stock = Column(Boolean, nullable=False)	Boolean availability value that must not be NULL.

Difference to remember: Item (SQLAlchemy model) represents the database table. ItemCreate and ItemResponse (Pydantic schemas) represent API input and output. The names are similar, but their jobs differ.

4. Database setup — models/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./items.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

DATABASE_URL

The URL "sqlite:///./items.db" tells SQLAlchemy to use a SQLite database file named items.db in the current working directory. SQLite stores data in a local file rather than requiring a separate database server.

create_engine(...)

Creates the SQLAlchemy engine, which knows how to communicate with the database. The SQLite option check_same_thread=False allows the connection to be used in FastAPI's typical request handling setup where work may occur across threads.

SessionLocal = sessionmaker(...)

Creates a factory for database sessions. A session is used to query, add, update and delete records. autocommit=False means changes are not automatically committed; the code explicitly calls commit(). autoflush=False disables automatic flushing before queries. bind=engine connects sessions to this engine.

Base = declarative_base()

Creates the base class inherited by SQLAlchemy models. SQLAlchemy collects model table definitions through Base.metadata.

get_db()

Creates a database session and yields it to the endpoint. The finally block closes the session after the request, even if an error occurs. FastAPI's Depends(get_db) manages this dependency lifecycle.

5. Item router — routers/itemsRouter.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import session
from models.database import get_db
from models.itemModel import Item
from models.itemSchema import ItemCreate, ItemResponse

router = APIRouter()

@router.post("/item", response_model=ItemResponse, status_code=201)
def create_Item(item: ItemCreate, db: session = Depends(get_db)):
    new_item = Item(
        name=item.name,
        price=item.price,
        in_stock=item.in_stock
    )
    db.add(new_item)
    db.commit()
    db.refresh(new_item)
    return new_item
```

APIRouter

Lets the item endpoints be grouped in a separate file instead of placing every endpoint in main.py.

Depends

FastAPI dependency injection. It tells FastAPI to call get_db and provide its yielded database session.

HTTPException

A FastAPI class used to return HTTP errors. It is imported here but not currently used in this endpoint.

session

The code uses this as a type annotation for db. In SQLAlchemy, the usual class name is Session with a capital S: `from sqlalchemy.orm import Session`, then `db: Session = Depends(get_db)`. This is a recommended cleanup.

Item, ItemCreate, ItemResponse

Item is the database model; ItemCreate validates the request body; ItemResponse defines the response shape.

router = APIRouter()

Creates a router object to which endpoint functions can be attached.

@router.post("/item", response_model=ItemResponse, status_code=201)

Registers a POST endpoint at /item. response_model validates/serialises the response. HTTP 201 means Created.

def create_Item(item: ItemCreate, db: session = Depends(get_db))

Defines the endpoint. FastAPI reads the JSON body into an ItemCreate object and injects a database session.

new_item = Item(...)

Creates a SQLAlchemy Item object using validated values from the request. It is not saved yet.

db.add(new_item)

Adds the new object to the session so SQLAlchemy knows it should be inserted.

db.commit()

Commits the transaction, causing the insert to be saved in the database.

db.refresh(new_item)

Refreshes the object from the database. This is useful for loading the generated id.

return new_item

Returns the database object. ItemResponse and from_attributes=True allow FastAPI/Pydantic to format it as JSON.

6. Application entry point — main.py

```
from fastapi import FastAPI
from models import itemModel
from models.database import Base, engine
from routers.studentsRouter import router as student_router
from routers.coursesRouter import router as course_router
from routers.itemsRouter import router as item_router

Base.metadata.create_all(bind=engine)

app = FastAPI()

@app.get("/")
def home():
    return {"msg": "this is main api home "}

app.include_router(student_router)
app.include_router(course_router)
app.include_router(item_router)
```

FastAPI() creates the application object. The home endpoint responds to GET /. The three include_router calls register the student, course and item routes with the app. Importing itemModel ensures the Item model is registered with SQLAlchemy metadata before Base.metadata.create_all(bind=engine) runs. That call creates mapped tables that do not already exist; it does not perform full schema migrations for changes to existing tables.

The import `from models import itemModel` is important in this setup because it loads the Item class before create_all is called. If a model has not been imported, SQLAlchemy may not know about its table yet.

7. Request flow: what happens when a client creates an item?

Step	What happens
1	Client sends POST /item with JSON fields name, price and in_stock.
2	FastAPI validates the body using ItemCreate. Invalid/missing fields normally produce HTTP 422.
3	Depends(get_db) opens a SQLAlchemy session and provides it to create_Item.
4	The router creates an Item object from the validated fields.

Step	What happens
5	db.add() stages the object; db.commit() saves it to items.db.
6	db.refresh() reads generated database values, including id, back into the object.
7	The endpoint returns the object; ItemResponse formats the output JSON.
8	get_db closes the session when the request finishes.

8. Test the endpoint in Swagger UI

1. Start the app from the project folder:

```
uvicorn main:app --reload
```

2. Open the interactive documentation in your browser:

```
http://127.0.0.1:8000/docs
```

3. Expand POST /item, click Try it out, and use this request body:

```
{
  "name": "Keyboard",
  "price": 1200,
  "in_stock": true
}
```

4. Click Execute. A successful request should return HTTP 201 and a response similar to the following (the id value is generated by the database):

```
{
  "id": 1,
  "name": "Keyboard",
  "price": 1200,
  "in_stock": true
}
```

If you submit a string for price or omit a required field, Pydantic validation will usually return HTTP 422. The exact validation details appear in the response.

9. Improvements and points to be aware of

• Use `from sqlalchemy.orm import Session` and annotate the dependency as `db: Session = Depends(get_db)`. The current import uses lowercase `session`, which is not the conventional SQLAlchemy session class and may cause a type/import issue depending on the installed SQLAlchemy version.

• Remove HTTPException from the imports if the router does not use it yet, or use it when handling errors such as an item not being found.

• Consider using `price: float` or `Decimal` if prices need fractional amounts. The current schema and database model use integers, so decimal prices are not represented.

• The current router only creates items. There are no GET, PUT or DELETE item endpoints in the retrieved itemsRouter.py file.

• For more robust transaction handling, consider rolling back if a database write fails. A failed transaction can otherwise leave the session in a failed state until it is closed or rolled back.

• Base.metadata.create_all creates missing tables, but it does not update existing table columns when the model changes. Use a migration tool such as Alembic if the project grows.

10. Quick glossary

Term	Simple meaning
API	A way for software applications to exchange data.
Endpoint	A URL plus an HTTP method, such as POST /item.
Schema	Rules describing the data accepted or returned by the API.
Model	The Python class mapped to a database table.
Router	A module that groups related endpoints.
Dependency injection	FastAPI supplies a required object, such as a database session, to a function.
Session	A unit of work used to interact with the database.
Commit	Save the current transaction to the database.
Refresh	Reload database-generated values into a Python object.
HTTP 201	Created: the request successfully created a resource.
HTTP 422	The request data failed validation.

This guide explains the item-related files retrieved from the repository's default branch. The repository also contains student and course routers; this document focuses on the item schema, model, router, database setup and the main application wiring.